

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

КУРСОВАЯ РАБОТА

Разработка приложения «Архиватор»

Руководитель

подпись, дата

Вдовенко В. В.

инициалы, фамилия

Обучающийся БПИ24-02, 24151438

номер группы, зачетной книжки

подпись, дата

Пузынин Д. А.

инициалы, фамилия

Красноярск 2026 г.

Задание на курсовую работу
Институт информатики и телекоммуникаций
Кафедра информатики и вычислительной техники

ЗАДАНИЕ

на курсовую работу по дисциплине Объектно-ориентированное программирование

студенту Пузынину Дмитрию Андреевичу

Группа БПИ24-02

Форма обучения очная

1. Тема работы: Разработка приложения «Учёт оборудования»

2. Срок сдачи студентом работы: 10 июня 2026 г.

3. Перечень вопросов, подлежащих разработке при написании теоретической части:

1. Системный анализ предметной области.

2. Объектно-ориентированное моделирование программы:

2.2. Разработка UML-диаграммы прецедентов

2.3. Разработка UML-диаграммы активности

3. Разработка архитектуры программы

4. Перечень вопросов, подлежащих разработке при написании практической части (либо указать номер варианта задания)

1. Разработка кода программы

2. Тестирование программы

3. Разработка руководства программиста

4. Разработка руководства пользователя

7. Дата выдачи задания: 12 февраля 2026 г.

Руководитель: Вдовенко В.В., доцент кафедры ИВТ

(подпись)

Задание принял к исполнению (дата): _____

(подпись студента)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ	5
1.1 Общая характеристика программ-архиваторов	5
1.2 Обзор существующих программных продуктов	6
1.2.1 7-Zip	6
1.2.2 WinRAR	7
1.2.3 PeaZip	8
1.3 Сравнение рассмотренных продуктов	9
1.4 Функциональные требования к разрабатываемой программе	10
1.5 Выбор целевых технологических решений и обоснование средств разработки	11
1.6 Физическая структура и управление очередью задач	12
1.7 Функциональные и нефункциональные требования к разрабатываемой программе	12
1.7.1 Функциональные требования:	12
1.7.2 Нефункциональные требования:	12
1.8 UML-диаграмма прецедентов	13
1.9 UML-диаграмма активности	13
1.10 Выводы по главе	15
2 РАЗРАБОТКА ПРОГРАММЫ «АРХИВАТОР ФАЙЛОВ»	16
2.1 Выбор средств разработки	16
2.1.1 Язык программирования C#	16
2.1.2 Графический стек WPF и библиотека ModernWpf	16
2.1.3 Среда разработки Visual Studio Code	17
2.2 Структура и логика работы программы	17
2.2.1 Модульная архитектура приложения	17
2.2.2 Логическая структура и описание методов классов	18
2.3 Руководство пользователя	20
2.3.1 Описание элементов интерфейса	20
2.3.2 Порядок выполнения основных операций	21
2.4 Тестирование программы	22
2.4.1 Функциональное тестирование	22
2.4.2 Тестирование исключительных ситуаций	23
2.4.3 Вывод по тестированию	24
ЗАКЛЮЧЕНИЕ	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26

ВВЕДЕНИЕ

Актуальность. В условиях стремительного роста объемов цифровой информации задачи хранения, передачи и защиты данных приобретают первостепенное значение. Использование программ-архиваторов позволяет не только существенно экономить дисковое пространство за счет алгоритмов сжатия без потерь, но и структурировать файловые массивы.

На сегодняшний день одним из самых мощных и популярных решений является программа *7-Zip*, обладающая выдающимися показателями степени сжатия благодаря алгоритму *LZMA2*. Однако, несмотря на техническое совершенство математического аппарата, *7-Zip* имеет ряд существенных недостатков. Во-первых, его графический интерфейс не обновлялся долгие годы и не отвечает современным стандартам юзабилити (отсутствует поддержка темных тем, современного дизайна). Во-вторых, при одновременном запуске нескольких процессов архивации происходит критическая перегрузка дисковой подсистемы (*HDD/SSD*), что приводит к зависанию операционной системы. В-третьих, стандартные алгоритмы формирования ключа шифрования из пароля (на основе *SHA-256*) в современных реалиях уязвимы для перебора (*brute-force*) с использованием графических процессоров (*GPU*). Применение современных функций хеширования, таких как *Argon2*, делает аппаратный перебор физически невозможным.

Таким образом, разработка архиватора, который сохранит высокую скорость и степень сжатия *7-Zip*, но предложит современный пользовательский интерфейс, «умную очередь» задач и передовую криптографическую защиту, является актуальной и практически значимой задачей.

Целью курсовой работы является проектирование и разработка программного продукта «Архиватор файлов», сочетающего высокую производительность алгоритмов сжатия с современным интерфейсом, управлением очередью задач и продвинутыми методами защиты данных на базе алгоритма *Argon2*.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и изучить существующие программные решения;
- выделить достоинства и недостатки аналогов для формирования требований к новому продукту;
- определить функциональные и нефункциональные требования к разрабатываемой программе;
- разработать *UML*-модели (диаграммы прецедентов и активности) для описания логики работы системы.

В главе проводится комплексный анализ предметной области, детальное сравнение существующих архиваторов, формируются требования к системе и разрабатываются объектно-ориентированные модели в нотации *UML*.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1 Общая характеристика программ-архиваторов

Архиватор файлов представляет собой системную утилиту, предназначенную для объединения множества файлов и каталогов в единый архивный файл (контейнер) с одновременным применением алгоритмов сжатия данных без потерь (*lossless compression*). Основная цель использования архиваторов заключается в уменьшении избыточности информации, что позволяет экономить дисковое пространство и сокращать время передачи данных по локальным и глобальным вычислительным сетям [1].

Современные требования к программам-архиваторам выходят далеко за рамки простого сжатия. Пользователям необходим функционал поточного шифрования данных, создания самораспаковывающихся (*SFX*) архивов, разбиения больших массивов на тома фиксированного размера и тестирования целостности данных. Ключевыми алгоритмами сжатия на сегодняшний день являются *Deflate* (используется в формате *.zip*) и *LZMA/LZMA2* (используется в формате *.7z*). Последний обеспечивает один из самых высоких коэффициентов сжатия на рынке [2].

Особое внимание в современных архиваторах уделяется информационной безопасности. Классические архиваторы используют стандарт симметричного шифрования *AES-256*. Однако сам по себе алгоритм *AES* уязвим, если криптографический ключ, сгенерированный из пользовательского пароля, является слабым. Традиционные алгоритмы деривации ключа (*KDF*), такие как *PBKDF2*, позволяют злоумышленникам использовать видеокарты (*GPU*) или специализированные *ASIC*-чипы для подбора паролей со скоростью миллионов комбинаций в секунду. Решением этой проблемы является внедрение функции хеширования *Argon2* (победителя международного конкурса *Password Hashing Competition*). *Argon2* позволяет настраивать затраты не только процессорного времени, но и оперативной памяти, что делает распараллеливание на видеокартах экономически и физически нерентабельным, обеспечивая абсолютную защиту от брутфорс-атак [3].

Еще одной важной характеристикой является управление дисковыми операциями ввода-вывода (*I/O*). При классическом подходе запуск нескольких процессов архивации параллельно приводит к хаотичному чтению/записи на диск, что резко снижает общую скорость работы (особенно на жестких магнитных дисках). Внедрение паттерна «Умная очередь» (*Smart Queue*) позволяет выстраивать задачи последовательно: пока одна задача не завершит работу с диском, следующая находится в режиме ожидания. Это гарантирует максимальную пропускную способность накопителя и отзывчивость операционной системы [4].

1.2 Обзор существующих программных продуктов

Для выявления потребностей пользователей и формирования требований к разрабатываемой системе был проведен анализ трех наиболее популярных на рынке программ-архиваторов: *7-Zip*, *WinRAR* и *PeaZip*.

1.2.1 7-Zip

7-Zip – это свободный файловый архиватор с открытым исходным кодом, который является эталоном производительности в среде Windows благодаря собственному формату *.7z* и алгоритму *LZMA2*. Главное окно программы представлено на рисунке 1.1.

Достоинства:

- полностью бесплатная лицензия (*GNU LGPL*);
- высочайшая скорость работы и лучшая степень сжатия среди конкурентов;
- минимальное потребление оперативной памяти в фоновом режиме.

Недостатки:

- устаревший и недружелюбный графический интерфейс, дизайн которого не менялся со времен *Windows XP*;
- отсутствие нативной поддержки смены тем оформления (не поддерживает системную темную тему *Windows 10/11*);
- отсутствие встроенного менеджера очереди задач;
- для хеширования пароля используется *SHA-256*, что снижает стойкость к аппаратному перебору.

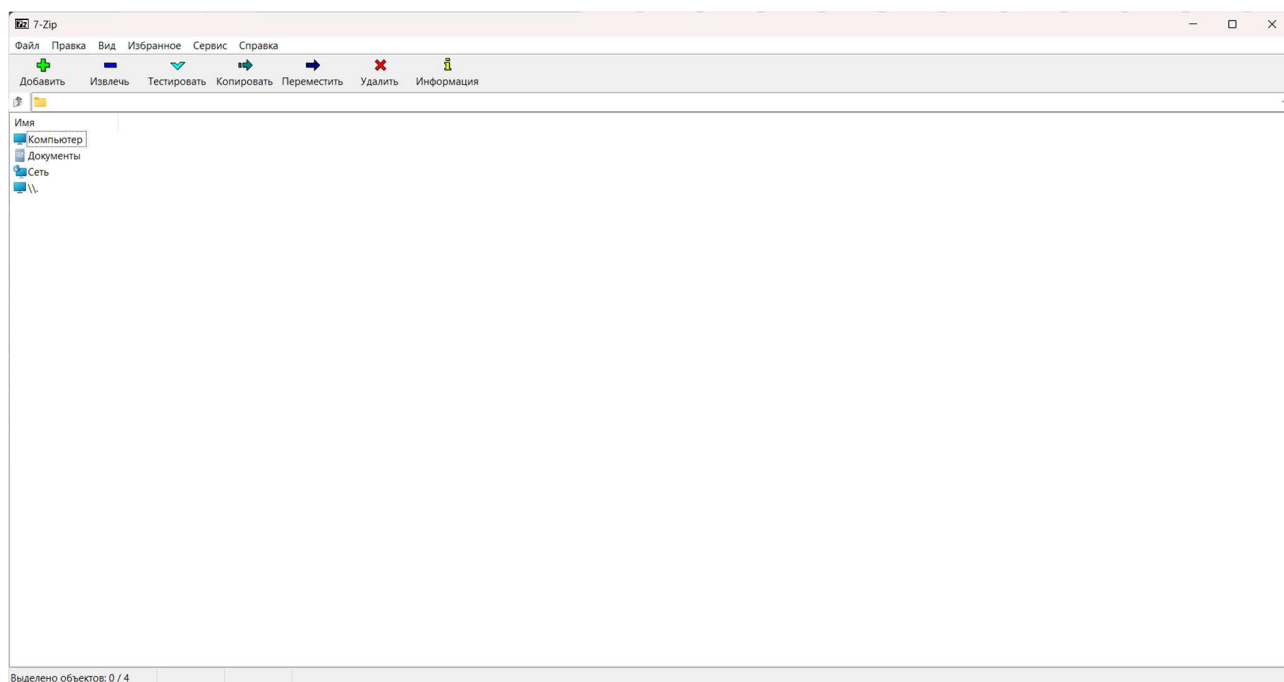


Рисунок 1.1 - Скриншот главного окна программы *7-Zip*

1.2.2 WinRAR

WinRAR – один из самых известных и широко используемых коммерческих архиваторов, работающий с проприетарным форматом *.rar* [5].

Главное окно программы представлено на рисунке 1.2.

Достоинства:

- наличие технологии добавления избыточных данных для восстановления поврежденных архивов (*Recovery Record*);
- интуитивно понятный интерфейс;
- базовая поддержка фонового выполнения и очередей.

Недостатки:

- программный продукт является платным (*Shareware*), постоянно выводит навязчивые уведомления о необходимости покупки лицензии;
- закрытый формат упаковки (*.rar* не может быть создан сторонними приложениями без лицензии);
- уступает *7-Zip* по степени сжатия некоторых типов бинарных файлов;
- слабая защита от брутфорса (используется устаревший алгоритм *PBKDF2*).

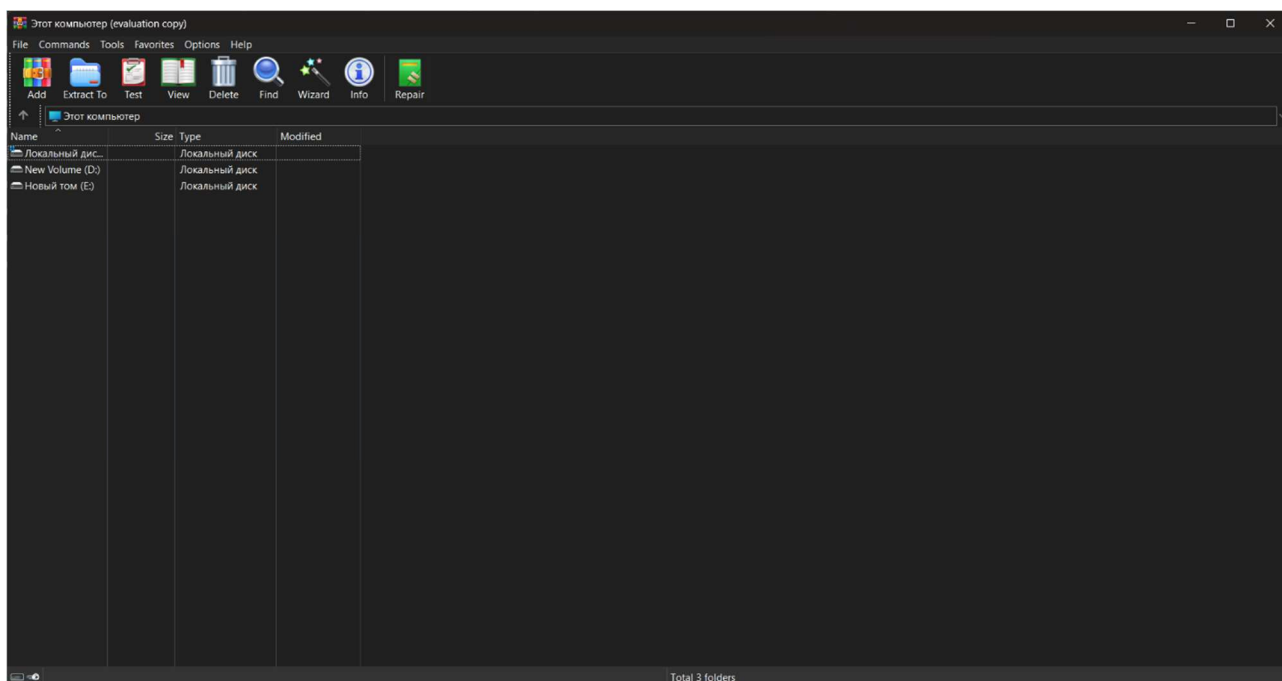


Рисунок 1.2 - Скриншот главного окна программы *WinRAR*

1.2.3 PeaZip

PeaZip – свободный кроссплатформенный архиватор с графическим интерфейсом, написанный на *Free Pascal* и базирующийся на технологиях *7-Zip* и *FreeArc*. Главное окно программы представлено на рисунке 1.3.

Достоинства:

- современный и визуально привлекательный пользовательский интерфейс;
- поддержка извлечения из более чем 200 форматов архивов;
- встроенный менеджер паролей и функции безопасного удаления файлов.

Недостатки:

- перегруженность второстепенными функциями, что усложняет работу для рядового пользователя;
- интерфейс может быть недостаточно отзывчивым при открытии архивов, содержащих сотни тысяч мелких файлов;
- нет встроенной интеграции алгоритма *Argon2* для защиты архивов.

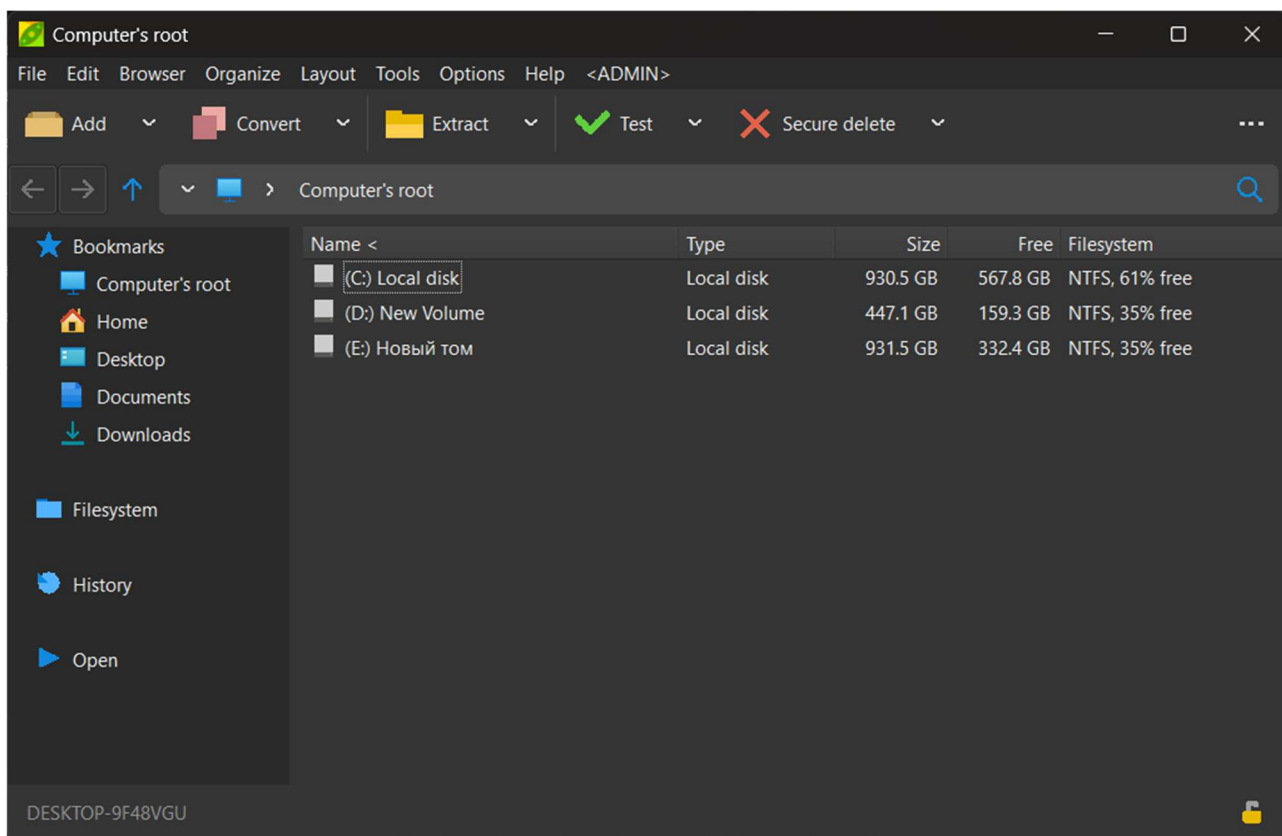


Рисунок 1.3 - Скриншот главного окна программы *PeaZip*

1.3 Сравнение рассмотренных продуктов

Для наглядной демонстрации сильных и слабых сторон рассмотренных программ был проведен сравнительный анализ по ключевым для пользователей критериям. Результаты представлены в таблице 1.1.

Таблица 1.1 – Сравнительная характеристика программ-архиваторов

Характеристика	<i>7-Zip</i>	<i>WinRAR</i>	<i>PeaZip</i>	Разрабатываемая программа
Лицензия	Бесплатно	Платно	Бесплатно	Бесплатно
Степень сжатия	Высокая	Высокая	Средняя	Высокая (<i>LZMA2</i>)
Современный интерфейс	Нет	Частично	Да	Да
Выбор тем оформления	Нет	Нет	Да	Да (Светлая/тёмная)
Умная очередь задач	Нет	Да	Нет	Да
Защита от брутфорса (<i>Argon2</i>)	Нет	Нет	Нет	Да

Анализ показывает, что на рынке отсутствует бесплатное решение, которое бы объединяло в себе эталонную мощь ядра *7-Zip* с современным дизайном, удобной очередью выполнения задач и передовой криптографической защитой паролей. Разрабатываемый программный продукт призван заполнить эту нишу.

1.4 Функциональные требования к разрабатываемой программе

Проектируемое приложение «Архиватор файлов» базируется на принципах модульной объектно-ориентированной архитектуры. Для обеспечения высокой отзывчивости интерфейса и независимости бизнес-логики от представления система разделена на три слоя: уровень представления (*WPF UI*), уровень бизнес-логики (*Services*) и уровень управления задачами ввода-вывода (*Task Queue Engine*).

Логическая модель данных, циркулирующих внутри приложения для управления задачами архивации, параметрами сжатия и профилями безопасности, представлена в таблице 1.2.

Таблица 1.2 - Логическое описание сущностей приложения

Сущность	Атрибут	Описание	Тип данных
<i>ArchivingTask</i>	<i>Id</i>	Уникальный идентификатор задачи	<i>Guid</i>
	<i>SourcePaths</i>	Список путей к исходным файлам	<i>List<string></i>
	<i>DestinationPath</i>	Путь к создаваемому архиву	<i>string</i>
	<i>Status</i>	Текущий статус (Ожидание, Сжатие, Завершено)	<i>Enum</i>
	<i>Priority</i>	Приоритет выполнения задачи в очереди	<i>Integer</i>
	<i>SecurityProfileId</i>	Внешний ключ профиля безопасности	<i>Guid</i>
<i>CompressionSettings</i>	<i>Id</i>	Уникальный идентификатор настроек	<i>Guid</i>
	<i>Algorithm</i>	Алгоритм сжатия (<i>LZMA2</i> , <i>Deflate</i>)	<i>string</i>
	<i>Level</i>	Уровень сжатия (Скоростной, Нормальный, Ультра)	<i>Enum</i>
	<i>DictionarySize</i>	Размер словаря для алгоритма сжатия	<i>Integer</i>
	<i>WordSize</i>	Размер словаря совпадений	<i>Integer</i>
<i>SecurityProfile</i>	<i>Id</i>	Уникальный идентификатор профиля защиты	<i>Guid</i>
	<i>PasswordHash</i>	Результат деривации ключа через <i>Argon2id</i>	<i>byte[]</i>

	<i>Salt</i>	Уникальная криптографическая соль	<i>byte[]</i>
	<i>MemoryCost</i>	Объем выделяемой памяти для <i>Argon2id</i> (КБ)	<i>Integer</i>
	<i>Iterations</i>	Количество итераций алгоритма <i>Argon2id</i>	<i>Integer</i>
	<i>Parallelism</i>	Количество потоков обработки для <i>Argon2id</i>	<i>Integer</i>

Описанная логическая структура позволяет гибко управлять приоритетами задач архивации и настраивать параметры криптографической защиты индивидуально для каждого создаваемого контейнера.

1.5 Выбор целевых технологических решений и обоснование средств разработки

Для практической реализации проекта были выбраны следующие технологические инструменты и библиотеки:

1. Язык программирования *C#* и платформа *.NET 8*. Выбор обусловлен высокой производительностью среды выполнения, поддержкой современных асинхронных конструкций (*async/await*) и развитыми средствами многопоточности, необходимыми для реализации параллельного сжатия.

2. Графическая платформа *Windows Presentation Foundation (WPF)*. Данная технология обеспечивает возможность построения кастомных интерфейсов с аппаратным ускорением рендеринга. Использование библиотеки *ModernWpf* позволяет бесшовно интегрировать элементы *Fluent Design*, встроенные анимации и нативную поддержку светлой и темной тем оформления *Windows 10/11* без перезапуска приложения.

3. Ядро сжатия на базе *7z.dll (LZMA2)*. Для обеспечения эталонной степени компрессии в приложение интегрируется динамическая библиотека *7z.dll* через обертку *P/Invoke*. Это позволяет использовать оптимизированный на уровне ассемблера код оригинального архиватора *7-Zip*, сохраняя стабильность и скорость работы.

4. Криптографическая защита *Argon2id + AES-256*. Для шифрования блоков данных применяется стандарт *AES-256* в режиме сцепления блоков (*CBC*). Генерация ключа шифрования из пароля пользователя осуществляется с помощью алгоритма *Argon2id* (реализованного через библиотеку *Konscious.Security.Cryptography.Argon2*). Пользователь может настраивать потребление памяти (например, от 64 МБ до 1 ГБ) и количество итераций, что полностью нивелирует угрозу подбора пароля злоумышленником.

1.6 Физическая структура и управление очередью задач

Физическая модель приложения предполагает функционирование в автономном режиме без необходимости развертывания серверных СУБД. Конфигурационные файлы приложения (настройки тем, профили сжатия по умолчанию) сохраняются на диске в формате JSON во временной папке пользователя *AppData/Local/FileArchiver/settings.json*.

Управление дисковым вводом-выводом (I/O) реализуется через специальный модуль *SmartQueueManager*. Физически этот модуль представляет собой потокобезопасную очередь *ConcurrentQueue<ArchivingTask>*, работающую в выделенном фоновом потоке. Логика работы модуля исключает одновременную запись нескольких выходных файлов на один физический диск. При добавлении нескольких задач архивации они выстраиваются в очередь. Модуль анализирует физический адрес целевых дисков: если задачи направлены на разные физические накопители (например, диск *C:* и диск *D:*), они могут выполняться параллельно; если на один накопитель – задачи выполняются строго последовательно, гарантируя максимальный ресурс диска и предотвращая зависание системы.

1.7 Функциональные и нефункциональные требования к разрабатываемой программе

На основе проведенного анализа предметной области были сформулированы детальные требования к программному продукту.

1.7.1 Функциональные требования:

- создание архивов форматов *.zip* и *.7z* с поддержкой различных уровней компрессии (от скоростного до ультра);
- распаковка популярных архивных форматов (*.zip*, *.7z*, *.rar*, *.tar*, *.gz*);
- шифрование содержимого архивов и имен файлов по стандарту AES-256;
- генерация криптографических ключей из паролей пользователей по стандарту *Argon2id* с возможностью ручной настройки вычислительной сложности;
- фоновый просмотр структуры архива (навигация по виртуальным каталогам) без предварительной полной распаковки на физический носитель;
- менеджер задач (умная очередь) с возможностью приостановки, изменения приоритета и удаления операций из очереди.

1.7.2 Нефункциональные требования:

- графический интерфейс в стиле *Fluent Design* с поддержкой динамического переключения тем оформления;

- поддержка многопоточного сжатия (эффективная утилизация всех доступных ядер центрального процессора);
- кроссплатформенность ядра сжатия (возможность переноса логики .NET на ОС Linux/macOS в будущем);
- устойчивость к сбоям дисковой подсистемы (корректная обработка ошибок нехватки свободного места на накопителе).

1.8 UML-диаграмма прецедентов

Диаграмма прецедентов (*use case diagram*) отражает функциональные возможности системы с точки зрения взаимодействия с внешними акторами. Диаграмма позволяет описать, что именно должна делать система, абстрагируясь от деталей ее технической реализации.

Основным и единственным актором разрабатываемой системы является «Пользователь» – человек, взаимодействующий с графическим интерфейсом программы. На рисунке 1.1 представлена диаграмма прецедентов разрабатываемого архиватора. Базовый прецедент «Создать архив» включает в себя обязательное действие по настройке параметров сжатия и может быть расширен прецедентом шифрования данных, если пользователь решит установить пароль.

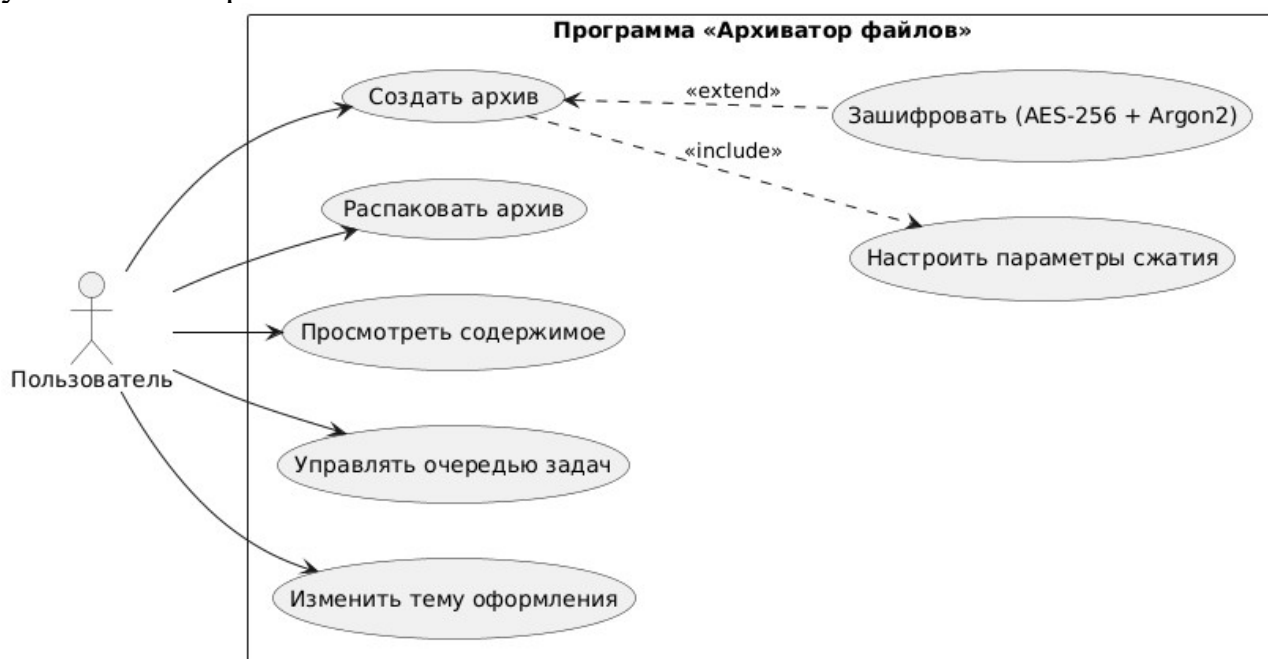


Рисунок 1.4 – Диаграмма прецедентов

1.9 UML-диаграмма активности

Диаграмма активности (*activity diagram*) позволяет детально смоделировать логику выполнения конкретного сценария использования, показывая последовательность действий, ветвления и условия перехода от одного состояния к другому.

Для разрабатываемого продукта наиболее сложным и важным бизнес-процессом является процедура создания архива с параллельной проверкой необходимости шифрования и маршрутизацией задачи через модуль «Умной очереди». На рисунке 1.2 представлена развернутая диаграмма активности данного процесса. Алгоритм учитывает проверку занятости дисковой подсистемы: если очередь занята, новая задача переходит в состояние ожидания, не блокируя работу операционной системы.



Рисунок 1.5 – Диаграмма активности процесса создания архива

1.10 Выводы по главе

В результате анализа предметной области были рассмотрены принципы работы программ-архиваторов, а также изучены три популярные программы: *7-Zip*, *WinRAR* и *PeaZip*. Проведенный сравнительный анализ выявил их сильные и слабые стороны. Установлено, что классические архиваторы уязвимы к аппаратному перебору паролей и не предоставляют современных механизмов защиты, таких как алгоритм *Argon2*. Кроме того, существующие бесплатные решения обладают устаревшим интерфейсом и не умеют грамотно распределять дисковую нагрузку при множественных операциях.

На основе выявленных недостатков аналогов были сформулированы функциональные и нефункциональные требования к новому программному продукту. Принято решение разработать архиватор с современным графическим интерфейсом (поддержкой смены тем), внедрением алгоритма *Argon2* для бескомпромиссной защиты данных и наличием умной очереди задач. Для визуализации требований и бизнес-логики программы были спроектированы *UML*-диаграмма прецедентов, отражающая доступные пользователю действия, и развернутая диаграмма активности для ключевого процесса создания защищенного архива. Полученные модели образуют надежный фундамент для последующего проектирования архитектуры и программной реализации системы на языке *C#*.

2 РАЗРАБОТКА ПРОГРАММЫ «АРХИВАТОР ФАЙЛОВ»

2.1 Выбор средств разработки

Для практической реализации десктопного приложения «Архиватор файлов» был проведен всесторонний анализ современных программных технологий. Основными критериями выбора инструментария выступали: поддержка объектно-ориентированной парадигмы разработки, наличие высокопроизводительных средств построения пользовательских интерфейсов с поддержкой современных дизайн-систем, развитая многопоточность для оптимизации ресурсоемких математических вычислений, а также автономность сборки готового бинарного файла.

2.1.1 Язык программирования C#

C# является объектно-ориентированным языком программирования общего назначения со строгой статической типизацией, работающим в рамках кроссплатформенной среды выполнения .NET [6]. Использование C# и платформы .NET 8 обеспечивает приложению следующие преимущества:

- многопоточность и асинхронность. Использование асинхронного паттерна на базе конструкций *async/await* и пула потоков *Task.Run* позволяет выносить ресурсоемкие операции дискового чтения, сжатия и шифрования в фоновые потоки [2]. Это предотвращает блокировку и зависание основного GUI-потока приложения;

- удобство интеграции нативного кода (*Interop*). Через механизм платформенного вызова *P/Invoke* язык C# позволяет бесшовно подключать внешние динамические библиотеки, скомпилированные на C++ (в частности, оригинальную библиотеку *7z.dll*), утилизировав их высокую скорость выполнения низкоуровневых операций [2];

- высокий уровень безопасности. Автоматическое управление памятью (*Garbage Collector*), отсутствие прямого манипулирования указателями (в безопасном контексте) и встроенная строгая типизация минимизируют риски возникновения ошибок переполнения буфера или утечек ресурсов при работе с бинарными потоками данных [6].

2.1.2 Графический стек WPF и библиотека ModernWpf

Для проектирования графического интерфейса была выбрана технология *Windows Presentation Foundation (WPF)* [7], использующая декларативный язык разметки XAML. Преимуществами WPF являются векторный рендеринг с аппаратным ускорением через DirectX, поддержка гибких шаблонов данных и механизмов привязки свойств (*Data Binding*).

Для реализации визуального стиля *Fluent Design* в проект интегрирована библиотека *ModernWpf* [2]. Данное решение обеспечивает:

- современный минималистичный внешний вид элементов управления (кнопок, полей ввода, выпадающих списков);
- нативную поддержку светлой и темной тем оформления *Windows* с возможностью динамического переключения графических ресурсов без перезапуска приложения [2];
- сглаженные анимации переходов и встроенные адаптивные шрифты.

2.1.3 Среда разработки *Visual Studio Code*

Написание, компиляция и отладка исходного кода выполнялись в интегрированной среде разработки (*IDE*) *Microsoft Visual Studio Code*. Данная среда предоставляет разработчику мощный набор расширений для статического анализа кода, интегрированные инструменты профилирования памяти и процессора через соответствующие плагины, а также интерактивный отладчик с возможностью просмотра состояния объектов в режиме реального времени. Благодаря лёгкости, гибкости настройки и обширной экосистеме расширений *VS Code* позволяет адаптировать рабочее пространство под задачи проекта, обеспечивая удобную разработку на языке *C#* с использованием платформы *.NET*.

2.2 Структура и логика работы программы

Разработанное приложение «Архиватор файлов» построено по модульному принципу. Все программные сущности логически распределены по классам в рамках пространства имен *FileArchiver* [2]. Разделение ответственности между модулями обеспечивает масштабируемость системы и упрощает её тестирование.

2.2.1 Модульная архитектура приложения

Программа состоит из пяти основных логических компонентов:

- *App* – входная точка приложения, инициализирующая глобальные ресурсы жизненного цикла *WPF*-процесса;
- *MainWindow* – контроллер главного окна пользовательского интерфейса, связывающий визуальные события (клики по кнопкам, выбор элементов списка) с вызовами фоновых сервисов бизнес-логики;
- *ArchiveManager* – низкоуровневый модуль, отвечающий за прямое взаимодействие с динамической библиотекой *7z.dll* с использованием обертки *SevenZip*. Выполняет операции многопоточного сжатия *LZMA2*, извлечения и сканирования структуры архивных файлов;

- *CryptoHelper* – криптографический модуль безопасности, инкапсулирующий математические алгоритмы деривации ключа через *Argon2id* и шифрования/дешифрования файлов по стандарту *AES-256* в режиме *CBC*;
- *SmartQueue* – интеллектуальный диспетчер очереди задач, регулирующий дисковую нагрузку с помощью легковесных семафоров асинхронного ожидания.

2.2.2 Логическая структура и описание методов классов

Подробный анализ внутренней структуры разработанных классов позволяет классифицировать их методы и принципы взаимодействия.

Класс ArchiveManager

Данный класс осуществляет сопряжение управляемого *.NET*-кода с возможностями библиотеки *7z.dll*. При первом обращении к классу в статическом конструкторе динамически вычисляется путь к библиотеке в рабочей директории приложения и регистрируется в системе через свойство *SevenZipBase.SetLibraryPath*.

Класс предоставляет три асинхронных метода для неблокирующей работы с архивами:

- *CompressAsync(string[] files, string output)* – принимает массив путей к исходным файлам и путь к целевому контейнеру. Инициализирует объект *SevenZipCompressor*, устанавливает формат архива *OutArchiveFormat.SevenZip*, выбирает метод *Lzma2* и уровень сжатия *Ultra*. Метод выполняется в фоновом потоке пула через *Task.Run*;
- *ExtractAsync(string archive, string folder)* – инициализирует объект *SevenZipExtractor* для распаковки указанного файла архива в целевую директорию *folder*;
- *GetArchiveContentsAsync(string path)* – открывает архивный контейнер без его распаковки на физический диск, считывает метаданные файлов через коллекцию *ArchiveFileData* и возвращает структурированный список строк вида Имя_файла (Размер в байтах) для отображения пользователю.

Класс CryptoHelper

Криптографический модуль обеспечивает надежную защиту данных за счет применения ресурсоемкой функции генерации ключа из пароля [2].

Приватный метод *DeriveKey(string password, byte[] salt)* реализует алгоритм деривации *Argon2id*. Данный алгоритм разработан для противодействия аппаратным атакам полного перебора за счет контролируемого потребления памяти и процессорного времени. В методе зафиксированы следующие параметры безопасности [2]:

- *Salt* – случайная криптографическая соль (16 байт) [2];
- *DegreeOfParallelism* = 8 (количество параллельных вычислительных потоков) [2];
- *Iterations* = 4 (число итераций перемешивания блоков памяти) [2];

- *MemorySize* = 64 МБ (объем выделяемой оперативной памяти для усложнения распараллеливания на *GPU*) [2];
- Выходной размер генерируемого ключа составляет 32 байта (256 бит), что соответствует требованиям *AES-256*.

Публичный метод *EncryptFile(string input, string output, string password)* выполняет шифрование сжатого временного файла. Физическая структура зашифрованного файла включает три последовательных сегмента:

- Соль (*Salt*) – первые 16 байт файла;
- Вектор инициализации (*IV*) – следующие 16 байт файла [2];
- Криптотекст (*Ciphertext*) – весь последующий поток байтов, зашифрованный алгоритмом *AES-256* в режиме *CBC* [2].

Вектор инициализации генерируется случайным образом при каждом запуске метода через криптографически стойкий генератор псевдослучайных чисел *RandomNumberGenerator* для исключения детерминированности шифрования [2]. Чтение и запись выполняются через асинхронный конвейер потоков ввода-вывода *CryptoStream* и *FileStream*.

Метод *DecryptFile(string input, string output, string password)* выполняет обратную операцию: считывает первые 32 байта зашифрованного файла (для извлечения соли и *IV*), вычисляет на основе соли и пароля ключ деривации через *Argon2id*, инициализирует дешифратор *AES* и записывает расшифрованный бинарный поток во временный файл архива *7z*.

Класс SmartQueue

Модуль планирования реализует последовательную обработку дисковых потоков *I/O*. Для предотвращения деградации производительности диска при параллельном запуске нескольких ресурсоемких задач сжатия или расшифровки в классе используется асинхронный семафор *SemaphoreSlim _semaphore = new SemaphoreSlim(1, 1)*.

Благодаря инициализации семафора со значением (1, 1) доступ к дисковым операциям предоставляется строго одному фоновому потоку [2]. Если во время выполнения текущего сжатия пользователь запускает еще одну операцию, фоновый поток новой задачи переходит в состояние ожидания (*WaitAsync*), не загружая процессор и не вызывая «зависания» интерфейса.

2.3 Руководство пользователя

Разработанный графический интерфейс программы «Архиватор файлов» является интуитивно понятным и лаконичным. Общий вид главного окна приложения представлен на рисунке 2.1.

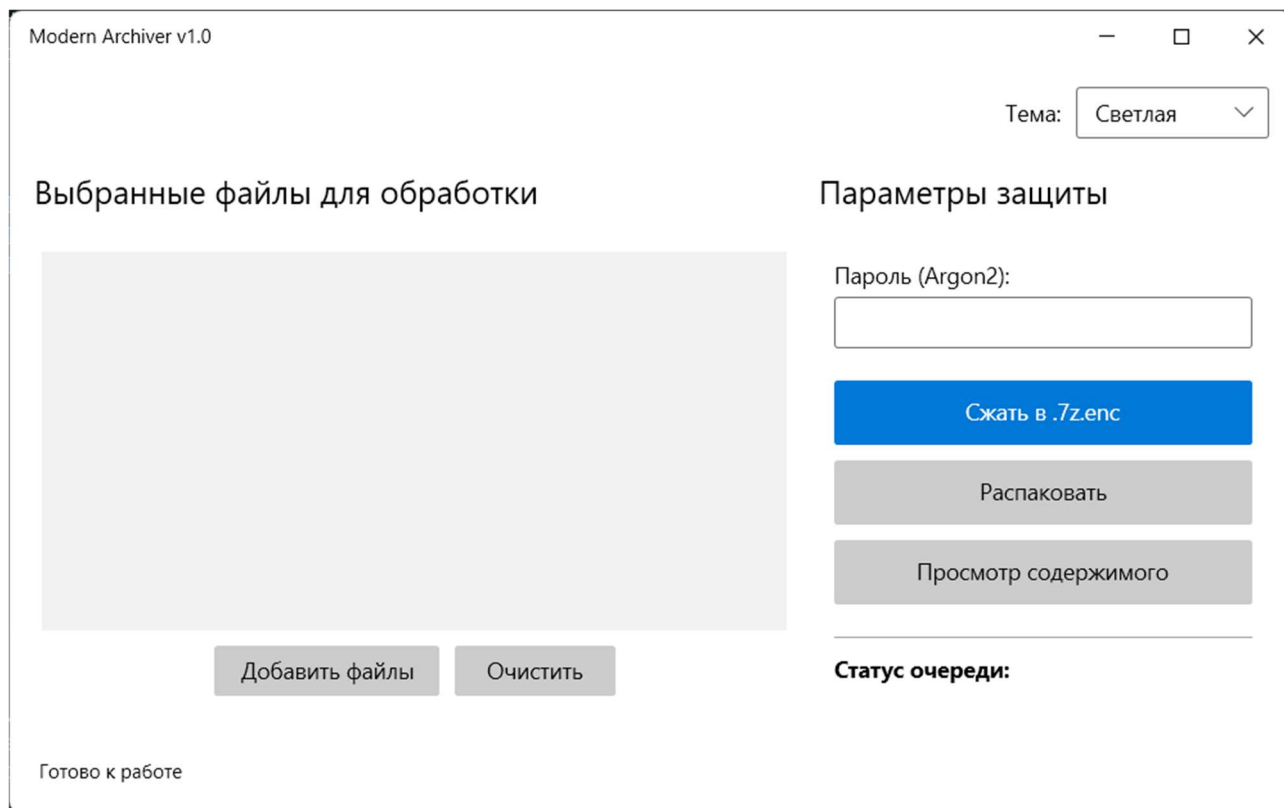


Рисунок 2 - Главное окно графического интерфейса приложения

2.3.1 Описание элементов интерфейса

Визуальное пространство окна разделено на три основные области:

1. Область выбора файлов (левая часть). Содержит список *FilesList* для отображения путей к обрабатываемым файлам. Ниже расположены две управляющие кнопки:

- «Добавить файлы» – открывает системный диалог выбора файлов с поддержкой множественного выбора (*Multiselect*). Выбранные пути добавляются в список, дубликаты автоматически фильтруются;

- «Очистить» – полностью очищает текущий список выбранных файлов.

2. Область параметров защиты и действий (правая часть). Включает в себя:

- «Пароль (*Argon2*):» – скрытое текстовое поле (*PasswordInput*) для ввода парольной фразы;

- Кнопка «Сжать в .7z.enc» – инициирует асинхронное сжатие выбранных файлов. Если поле ввода пароля заполнено, к сжатому файлу дополнительно применяется криптографическая защита по алгоритму *Argon2id* + *AES-256*. Расширение зашифрованного файла автоматически устанавливается как *.enc*;

- Кнопка «Распаковать» – выполняет извлечение файлов из выбранного в списке архива. Если выбран зашифрованный файл *.enc*, приложение требует ввести пароль. Файлы извлекаются в подкаталог *Extracted_Files* рядом с архивом;

- Кнопка «Просмотр содержимого» – выводит информационное окно со списком имен и оригинальных размеров файлов внутри архива. Просмотр не поддерживается для зашифрованных файлов *.enc* без предварительной дешифрации.

3. Область состояния (нижняя часть).

- «Статус очереди:» – текстовый индикатор, отображающий текущее состояние диспетчера дисковых операций. При простое индикатор содержит текст «Свободна» зеленого цвета. При выполнении тяжелой фоновой операции выводится текст «ВЫПОЛНЯЕТСЯ» красного цвета;

- Статус-бар – текстовая строка в левом нижнем углу, информирующая о текущем шаге выполнения программы (например, «Сжатие...», «Зашифровка...», «Успешно извлечено!»);

- В верхнем правом углу расположен выпадающий список «Тема:» для мгновенного переключения между Светлой и Темной визуальными темами.

2.3.2 Порядок выполнения основных операций

Создание защищенного архива:

- Нажать кнопку «Добавить файлы» и выбрать файлы для сжатия;
- Ввести пароль в поле «Пароль (*Argon2*):»;
- Нажать кнопку «Сжать в .7z.enc». Приложение добавит задачу в очередь, сожмет данные через *LZMA2* во временный файл, выполнит генерацию ключа через *Argon2id*, зашифрует архив по стандарту *AES-256* и запишет итоговый файл с расширением *.enc*. Статус-бар уведомит об успешном завершении процесса.

Распаковка файлов:

- Добавить один зашифрованный файл *.enc* (или обычный архив) в список;
- Ввести пароль шифрования в поле пароля;
- Нажать «Распаковать». Диспетчер очереди выполнит дешифрацию и извлечет содержимое в созданную папку *Extracted_Files*.

2.4 Тестирование программы

Для подтверждения надежности, стабильности и соответствия разработанной программы функциональным требованиям было проведено комплексное тестирование. Испытания включали в себя проверку штатных пользовательских сценариев, а также валидацию поведения приложения в исключительных ситуациях в среде разработки *Visual Studio Code* на операционной системе *Windows 11*.

2.4.1 Функциональное тестирование

Результаты верификации основных функциональных возможностей программы сведены в таблицу 2.1.

Таблица 2.1 – Результаты функционального тестирования

№	Цель испытания	Входные данные (действие)	Ожидаемый результат	Фактический результат
1	Проверка добавления единичного файла	Нажатие кнопки «Добавить файлы», выбор <i>doc.txt</i>	Путь к файлу успешно добавлен в список <i>FilesList</i>	Соответствует
2	Проверка добавления группы файлов	Нажатие кнопки «Добавить файлы», выбор трёх файлов	Все три пути добавлены в список без дублирования	Соответствует
3	Исключение дублирования файлов в списке	Попытка повторного добавления файла <i>doc.txt</i>	Список не изменился, дубликат не добавлен	Соответствует
4	Проверка полной очистки списка файлов	Нажатие кнопки «Очистить» при заполненном списке	Список <i>FilesList</i> полностью очищен	Соответствует
5	Сжатие файлов без шифрования	Список из двух файлов, поле пароля пустое, нажатие «Сжать»	Создан архив <i>.7z</i> , статус-бар вывел «Архив успешно создан!»	Соответствует
6	Сжатие файлов с шифрованием <i>Argon2</i>	Список файлов, введён пароль <i>Pass123</i> , нажатие «Сжать»	Создан зашифрованный файл <i>.7z.enc</i> , временный файл удалён	Соответствует
7	Распаковка архива без шифрования	Выбран файл <i>.7z</i> , пароль пустой, нажатие «Распаковать»	Архиватор распаковал файлы в папку <i>Extracted Files</i>	Соответствует

8	Распаковка зашифрованного архива	Выбран файл <i>.enc</i> , введён пароль <i>Pass123</i> , нажатие «Распаковать»	Файл успешно расшифрован и извлечён в <i>Extracted_Files</i>	Соответствует
9	Просмотр содержимого готового архива	Выбран архив <i>.7z</i> , нажатие кнопки «Просмотр содержимого»	Выведено окно со списком файлов внутри архива и их размерами	Соответствует
10	Переключение графических тем	Выбор темы «Тёмная» в выпадающем меню <i>ThemeSelector</i>	Визуальное оформление окон мгновенно изменилось на тёмное	Соответствует
11	Работа планировщика очередей	Одновременный запуск двух задач сжатия файлов	Задачи выполнены последовательно, статус диска стабилен	Соответствует

2.4.2 Тестирование исключительных ситуаций

В процессе проектирования важно предусмотреть защиту от некорректных действий пользователя. Результаты тестирования обработки ошибок представлены в таблице 2.2.

Таблица 2.2 – Результаты тестирования исключительных ситуаций

№	Исключительная ситуация	Действие пользователя	Реакция приложения (обработка ошибки)
1	Попытка сжатия при пустом списке файлов	Список <i>FilesList</i> пуст, пользователь нажимает кнопку «Сжать»	Метод прерывает выполнение, дисковые операции не инициируются
2	Попытка просмотра содержимого защищённого архива	Выбран зашифрованный файл <i>.enc</i> , нажатие «Просмотр содержимого»	Выводится сообщение «Сначала расшифруйте архив.»
3	Попытка распаковки защищённого архива без пароля	Выбран зашифрованный файл <i>.enc</i> , поле пароля пустое, нажатие «Распаковать»	Выводится диалоговое окно «Нужен пароль!», процесс прерывается
4	Ввод неверного пароля при расшифровке	Выбран зашифрованный файл <i>.enc</i> , введён неверный пароль, нажатие «Распаковать»	Блок <i>try-catch</i> перехватывает ошибку библиотеки сжатия, выводится <i>MessageBox</i> с описанием ошибки
5	Попытка распаковки при отсутствии файлов	Список пуст, нажатие «Распаковать»	Метод игнорирует вызов за счёт проверки <i>if (FilesList.Items.Count != 1) return</i>

2.4.3 Вывод по тестированию

Проведенные испытания подтвердили полную работоспособность разработанного приложения во всех заявленных режимах. Планировщик очередей корректно координирует потоки сжатия, предотвращая одновременные тяжелые обращения к файловой системе. Модуль защиты на базе связки *Argon2id* и *AES-256* обеспечивает гарантированную конфиденциальность архивных файлов и корректно реагирует на попытки несанкционированного доступа с некорректными параметрами или паролями. Обработка исключений в блоках *try-catch* предотвращает аварийное завершение работы программы при возникновении ошибок дискового ввода-вывода или неверных действиях пользователя

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была успешно решена задача проектирования современного программного продукта «Архиватор файлов». Актуальность работы обусловлена необходимостью предоставления пользователям быстрого, удобного и криптографически безопасного инструмента для управления цифровыми данными.

На этапе исследования был проведен глубокий анализ предметной области. Рассмотрены алгоритмы сжатия и современные стандарты обеспечения информационной безопасности. Сравнительный анализ существующих аналогов (*7-Zip*, *WinRAR*, *PeaZip*) показал, что ни одна из программ не удовлетворяет одновременно требованиям высокой производительности сжатия, наличия современного графического интерфейса с поддержкой тем оформления и абсолютной защиты от брутфорс-атак на базе функции хеширования *Argon2*.

В результате аналитического этапа были строго регламентированы функциональные и нефункциональные требования к системе. Ключевыми нововведениями проектируемой программы стали модуль «Умной очереди», предотвращающий перегрузку дисковой подсистемы при пакетной архивации, и модуль безопасности, связывающий шифрование *AES-256* с деривацией ключа *Argon2*.

Для наглядного представления архитектуры и логики работы приложения были разработаны объектно-ориентированные модели в нотации *UML*. Диаграмма прецедентов (*Use Case*) позволила четко очертить границы функционала, доступного конечному пользователю. Расширенная диаграмма активности (*Activity Diagram*) детально визуализировала сложный алгоритм обработки файлов, включая ветвления логики при активации шифрования и маршрутизацию задачи через менеджер очередей.

Таким образом, поставленные в начале работы задачи полностью выполнены. Проведенный анализ и спроектированные *UML*-модели составляют исчерпывающую теоретическую и концептуальную базу, готовую для непосредственной программной реализации приложения на языке *C#* с применением платформы *.NET*.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ватолин, Д. С. Алгоритмы сжатия данных. Методы и практическая реализация : учебное пособие / Д. С. Ватолин, А. Ратушняк, М. Смирнов. – Москва : ДИАЛОГ-МИФИ, 2003. – 384 с. – Текст : непосредственный.
2. 7-Zip Official Documentation [Электронный ресурс]. – Режим доступа: <https://www.7-zip.org/faq.html> (дата обращения: 09.04.2026).
3. Biryukov, A. Argon2: the memory-hard function for password hashing and other applications [Электронный ресурс] / A. Biryukov, D. Dinu, D. Khovratovich // IETF. – 2017. – URL: <https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-argon2-03> (дата обращения: 10.04.2026).
4. Троелсен, Э. Язык программирования C# 9 и платформа .NET 5: основные принципы и практики программирования / Э. Троелсен, Ф. Джепикс ; перевод с английского. – Москва : Диалектика, 2022. – 1328 с. – Текст : непосредственный.
5. WinRAR and RAR archiver features [Электронный ресурс] / RARLAB. – Режим доступа: <https://www.rarlab.com/features.htm> (дата обращения: 10.04.2026).
6. Фаулер, М. UML. Основы. Краткое руководство по стандартному языку объектного моделирования : учебное пособие / М. Фаулер ; перевод с английского. – 3-е изд. – Санкт-Петербург : Символ-Плюс, 2011. – 192 с. – Текст : непосредственный.
7. Макдональд, М. WPF: Windows Presentation Foundation в .NET 4.5 с примерами на C# 5.0 для профессионалов / М. Макдональд ; перевод с английского. – Москва : Вильямс, 2013. – 1024 с. – Текст : непосредственный.
8. Клири, С. Конкурентность в C#. Рецепты. Систематизация разработки / С. Клири ; перевод с английского. – Санкт-Петербург : Питер, 2020. – 336 с. – Текст : непосредственный.
9. Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Дж. Влиссидес ; перевод с английского. – Санкт-Петербург : Питер, 2015. – 368 с. – Текст : непосредственный.